

TRANSFORMER DESDE CERO

Attention is all you need

De un bigrama a un Transformer: construir un modelo de lenguaje desde cero



El paper: Attention Is All You Need

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

REFERENCIA

Vaswani, Shazeer, Parmar, Uszkoreit, Jones, Gomez, Kaiser & Polosukhin

NeurIPS 2017 · arXiv:1706.03762

Introdujo el Transformer: una arquitectura basada enteramente en atención, sin recurrencia ni convoluciones. Es la base de los modelos de lenguaje actuales.

En esta clase lo reconstruimos desde el modelo de lenguaje más simple posible, pieza por pieza.

El Transformer del paper

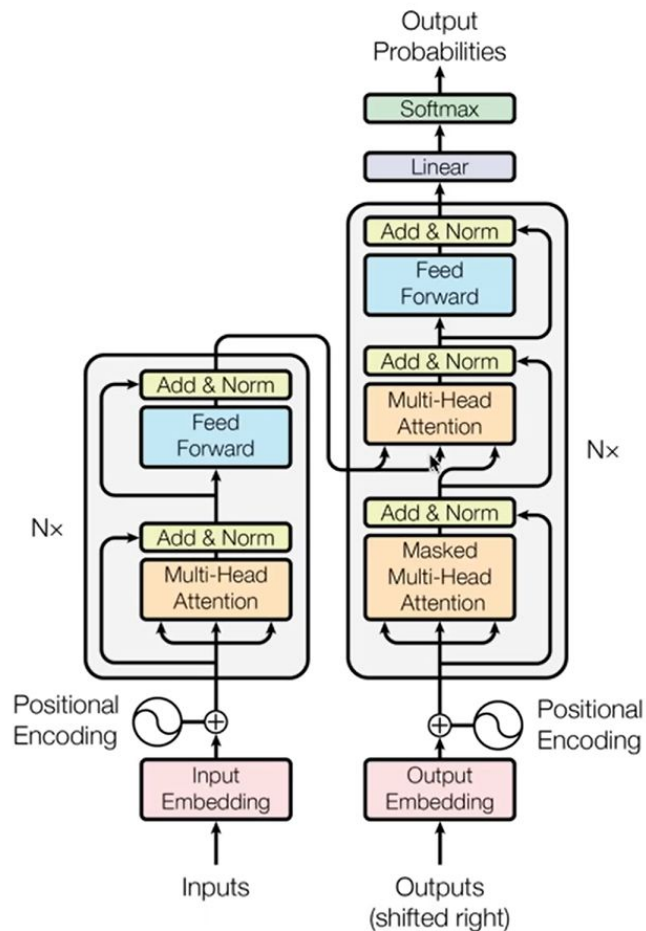


Figura 1 — Vaswani et al. (2017)

La arquitectura original es un encoder-decoder — dos mitades — pensado para traducción automática.

Hoy vamos a construir y entender cada pieza de esta figura, empezando por el modelo más simple posible.

Al final volvemos a esta figura para ubicar exactamente qué parte implementamos (y qué queda afuera).

Del texto a números: tokens y vocabulario

Tokenizar. partir el texto en unidades de un conjunto finito. Hay tres granularidades:

carácter

g · a · t · o

subpalabra

gat · o

palabra

gato

Vocabulario. el conjunto de todos los tokens posibles; su tamaño es V .

Hoy trabajamos a nivel de caracteres — por eso V es chico:

a b c d e $V = 5$

Aviso de orden del curso. en esta materia vemos primero el Transformer y después, en detalle, cómo se tokeniza y cómo se pasa de token a vector (embeddings). Hoy alcanza con la intuición: texto $\rightarrow$ tokens de un vocabulario finito.

Caracteres vs palabras: un trade-off

	vocabulario	longitud de secuencia	fuera de vocab.
por carácter	chico	larga	no
por palabra	enorme	corta	sí
subpalabra (BPE)	medio	media	no

mismo texto “gato”, en las dos puntas:

carácter	g a t o	→ 4 tokens · vocab chico
palabra	gato	→ 1 token · vocab enorme

El tira y afloje. carácter = vocab chico pero secuencias largas; palabra = vocab enorme pero secuencias cortas. La subpalabra busca lo mejor de ambos.

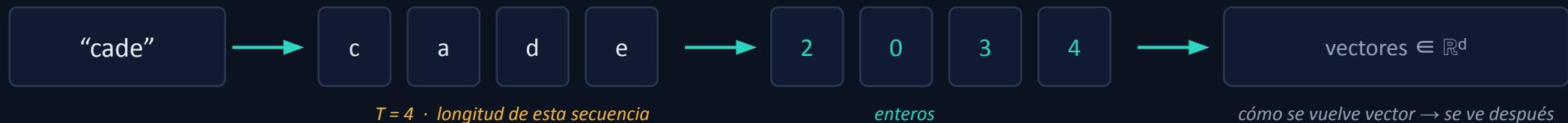
las secuencias largas se pagan caro — lo vemos en la Parte 3 (costo de la atención).

Dos tamaños que no hay que confundir: V y T

V — tamaño del vocabulario. cuántos símbolos distintos EXISTEN. Es fijo para todo el modelo.

T — longitud de la secuencia. cuántos tokens tiene ESTE texto. Varía de un texto a otro.

ejemplo con el vocabulario a b c d e:



El puente con la Clase 1. aquella clase asumía que la entrada ya era un vector. Acá aparece lo que faltaba: texto $\rightarrow$ número (id) $\rightarrow$ vector. La atención opera sobre T; el vocabulario define V.

Qué vamos a construir

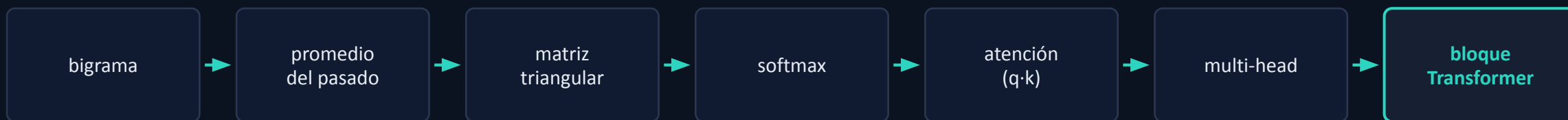
El objetivo. Un modelo de lenguaje a nivel de caracteres — que predice el próximo carácter — empezando por el más simple posible y mejorándolo, paso a paso, hasta llegar a la atención y al Transformer.

La regla del día. construir, no postular: cada pieza nueva aparece porque la anterior se queda corta.

Cómo lo vemos. cada mejora se acompaña de dos cosas: un ejemplo numérico chico (vocabulario de tamaño $V=5$) para ver bien las estructuras, y su implementación real en la notebook (con el vocabulario completo).

La progresión de hoy

Cada flecha es una mejora concreta sobre la anterior. Volvemos a este mapa en el cierre.

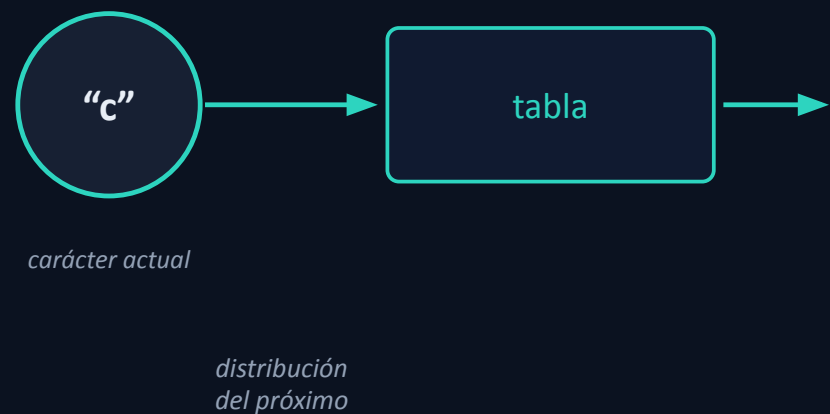


En cada paso: baja el loss del modelo, y la estructura del texto que genera se parece cada vez más a un guion — aunque el sentido siga siendo nulo.

El modelo más simple posible

Bigrama. predecir el próximo carácter mirando **únicamente el carácter actual** — sin contexto, sin historia.

Es el primer intento honesto de un modelo de lenguaje. Su simpleza es justamente el punto de partida: enseguida vamos a ver, con números, qué le falta — y esa falta motiva todo lo que viene.



La tabla de lookup

```
token_embedding_table =  
    nn.Embedding(V, V)
```

El modelo entero es esta tabla. La fila i son los logits del próximo token, dado el token i . Por eso es $V \times V$: una dimensión elige la fila (token actual), la otra recorre los candidatos a próximo token.

Sin entrenar todavía: la tabla arranca con valores random ($N(0,1)$).

próximo token →		a	b	c	d	e
token actual	a	-0.70	-0.49	-0.32	-1.76	0.21
	b	-2.01	-0.56	0.34	1.55	-1.37
	c	1.43	-0.28	-0.56	1.19	1.70
	d	-1.69	-0.70	0.58	0.98	-1.22
	e	-1.33	0.00	-1.32	-0.38	1.27

la fila de "c" = lo que el modelo predice después de una c

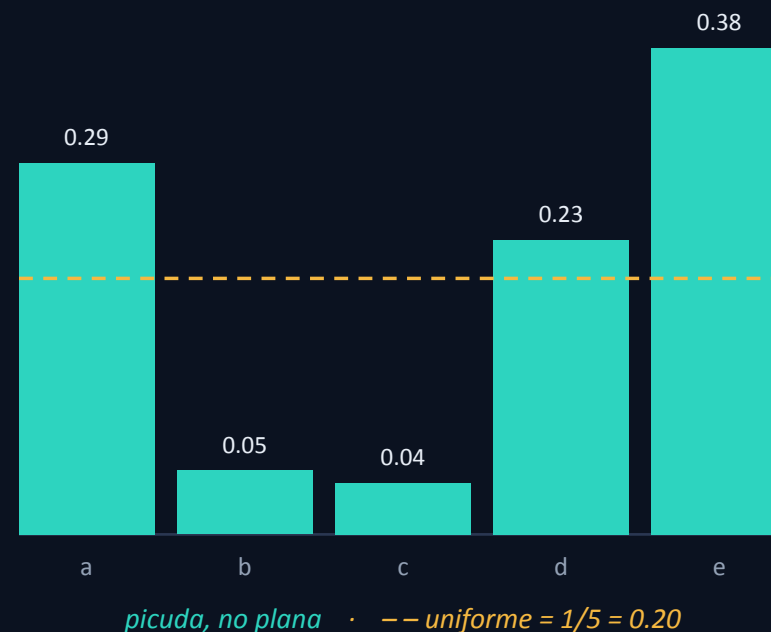
De logits a probabilidades

$\text{softmax}(\text{fila}_c) = [0.29, 0.05, 0.04, 0.23, 0.38]$

Para convertir la fila en probabilidades, aplicamos softmax. La distribución es picuda, no uniforme: el modelo apuesta por “e” (0.38) y “a” (0.29) y casi descarta “c” (0.04) — sin ninguna razón, porque los números son random.

La clave. esa confianza arbitraria — apostar fuerte por tokens equivocados — es lo que explica el loss antes de entrenar (próxima lámina).

distribución softmax de la fila “c”



¿Cómo hay loss antes de entrenar?

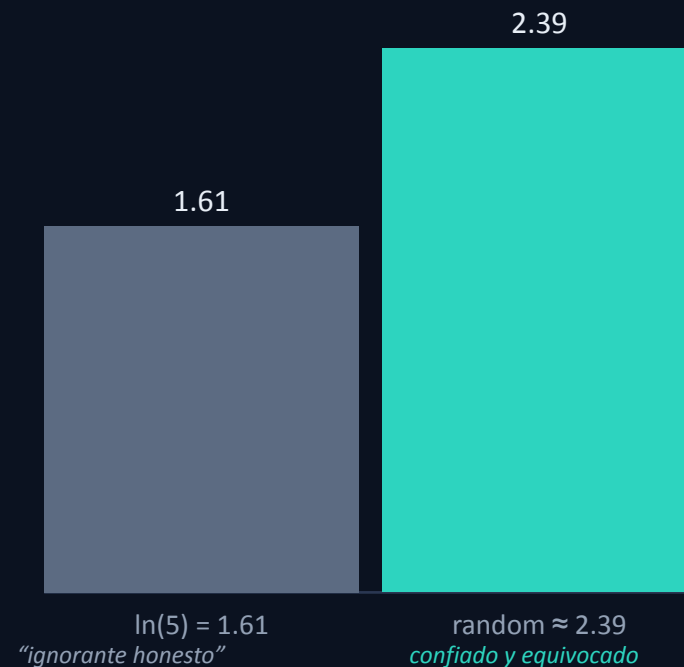
La idea clave. el loss es una **medición**, no un resultado del entrenamiento. Se puede calcular para cualquier tabla — incluso una random.

loss = promedio de $-\log p(\text{token correcto})$

La tabla arranca con números random, así que ya produce predicciones (malas). El loss mide cuán lejos están de los próximos caracteres reales del corpus. Entrenar es lo que después baja ese número.

El detalle fino. un modelo que no supiera nada daría $1/V$ a cada token → loss = $\ln(V)$. Pero la tabla random es picuda y equivocada, así que saca un loss MÁS ALTO que ese piso.

loss: piso uniforme vs tabla random



Entrenar un bigrama = contar

gradiente en la fila del token actual: $\text{grad} = p - \text{onehot}(\text{correcto})$

Sube el correcto, baja el resto. el gradiente es negativo en el token correcto (lo empuja hacia arriba) y positivo en los demás (los baja). Es toda la regla de aprendizaje.

Y como cada token recibe gradiente de todas las posiciones donde aparece, **su fila converge a la distribución empírica de qué sigue a ese carácter.** Entrenar un bigrama es, literalmente, contar pares de caracteres.

un paso sobre la fila "c" (supongamos que el correcto = d):

a	b	c	d	e
0.29	0.05	0.04	0.23	0.38

antes

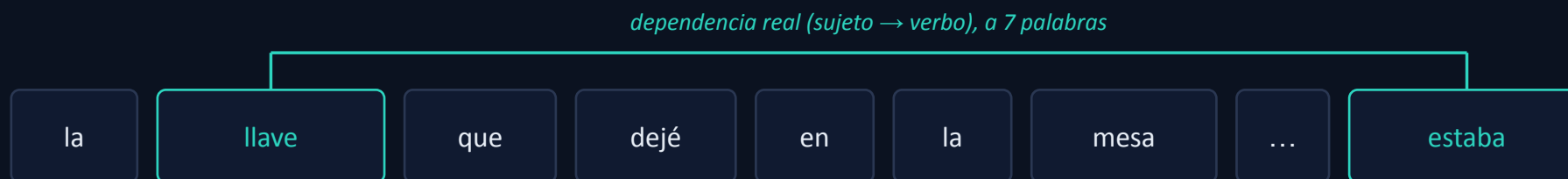
a	b	c	d	e
0.21	0.05	0.04	0.47	0.25

después: $p(d)$ subió $0.23 \rightarrow 0.47$

Podés saltar el gradiente: tabular las frecuencias de pares da la misma tabla. El optimizador solo descubre los conteos.

El techo del bigrama

La pared. softmax(fila c) sólo puede codificar “qué caracteres suelen seguir a c”, promediado sobre todo el corpus. No puede decir “esta c, en este contexto, predice distinto” — porque no conoce el contexto.



✗ *el bigrama sólo ve la palabra inmediatamente anterior — ciego a «llave»*

Lo que falta: darle al modelo memoria del pasado — no sólo el token anterior. Esa es la Parte 2.

► **VER IMPLEMENTACIÓN (notebook)** entrenar el bigrama y generar. Loss $\approx \ln(65)$. Muestra: caracteres con frecuencias correctas, sin estructura.

¿Qué genera el bigrama?

Entrenado el bigrama, generamos muestreando carácter a carácter de la fila correspondiente.

SALIDA GENERADA · notebook (V=65)

```
Cprs an tcede.
YEIn, lroloundee waindonse ate t,-bee wist ic wsoster; bea yonsenimser se ay g pourancey mou ber s LI'sl tem'ls tofr?

KIVod, IAg thorvere nonifit deanver
Whatrerath;3f t
wise pls tode ish rithoo bsin henghimblyondiOLe;;
S:

Hennsen l'Ve ph POLURI af, ty V: gill.
SCly a au a owe hotemanven.
Y:
HANouro asthoptong,
BThavithiritouj: howie sed hathitious; d healol lemek'le. shil?
Thatiswo amanghen ivethe:
IOKA:
awh if clend ce ifth reris r blithout yoe n,
T:

TIUze bld, t kechug t, nt
```

Qué mirar. no el sentido — la estructura. Salen caracteres con frecuencias plausibles (espacios, vocales), pero sin palabras ni líneas.

Por qué. el modelo sólo conoce el carácter inmediatamente anterior, así que no hay forma de que aparezca estructura de palabras o de guion.

Éste es el piso contra el que comparamos cada mejora.

Darle memoria: promediar el pasado

La idea. la nueva representación de cada token = el **promedio** de todos los tokens hasta él (incluido él mismo). Un “bag of words” del pasado — burdo, pero es el punto de partida.

Ojo causal. sólo promediamos hacia atrás, nunca el futuro — verlo sería hacer trampa. Lo construimos de tres formas equivalentes, cada vez más “elegante”.



cada token recibe de todos los anteriores...

...con el MISMO peso (promedio uniforme)

Versión 1 — el for-loop

```
for b in range(B):
    for t in range(T):
        xbow[b,t] = x[b, :t+1].mean(0)
```

xbow[t] = promedio de x[0..t]. la forma directa: un doble for.

Correcto, pero lento. dos loops en Python. La pregunta natural: ¿podemos hacer esto de una sola vez, en paralelo?

ejemplo (T=4, C=2):

x			xbow	
1	0	→	1.00	0.00
2	2		1.50	1.00
4	1		2.33	1.00
0	3		1.75	1.50

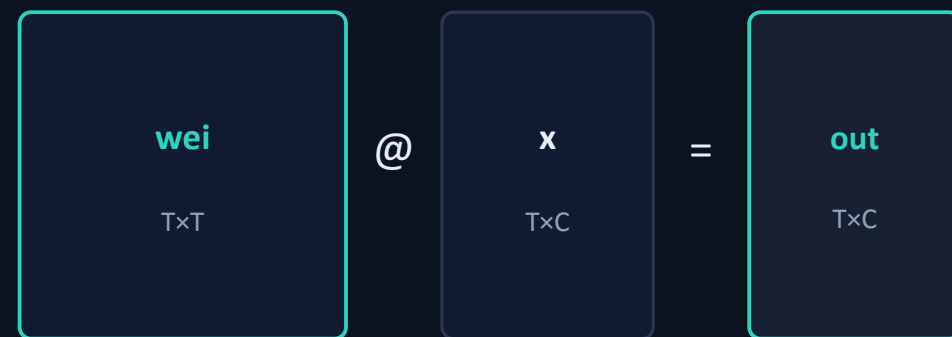
cada fila = media acumulada de las de arriba

La idea clave: agregación ponderada = multiplicar matrices

$$\text{salida} = \text{wei} @ \mathbf{x}$$

El truco a internalizar. cualquier agregación ponderada de los tokens es una multiplicación de matrices: una matriz wei de $T \times T$, donde la fila i dice con qué peso entra cada token en la salida de la posición i .

Reaparece hasta el final. todas las posiciones se calculan de una, en paralelo. La pregunta: ¿qué wei reproduce el promedio del pasado?



fila i de wei = la mezcla para la posición i

Versión 2 — matriz triangular normalizada

```
wei = torch.tril(torch.ones(T,T))  
wei = wei / wei.sum(1, keepdim=True)
```

La fila i reparte peso $1/(i+1)$ entre las posiciones $0..i$, y cero al futuro (el triángulo superior es cero).

Idéntico al for-loop. $\text{xbow2} = \text{wei} @ \text{x}$ da exactamente las mismas medias acumuladas — pero en un solo matmul, paralelo.

wei (triangular inferior, uniforme por fila)

1.00	0.00	0.00	0.00
0.50	0.50	0.00	0.00
0.33	0.33	0.33	0.00
0.25	0.25	0.25	0.25

cada fila suma 1 · el futuro pesa 0

Versión 3 — softmax de afinidades

```
wei = torch.zeros((T,T))
wei = wei.masked_fill(
    tril==0, float('-inf'))
wei = F.softmax(wei, dim=-1)
```

Parece un rodeo, pero es la que abre la puerta a la atención.

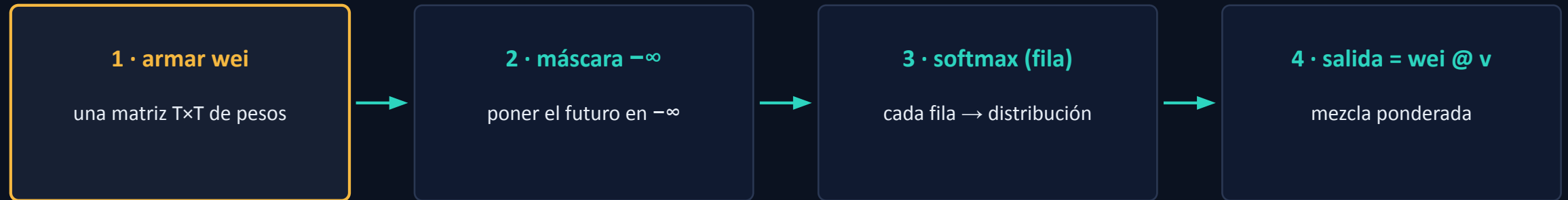
Dos trabajos separados. el $-\infty$ es la máscara causal ($\exp(-\infty)=0$ mata el futuro); los zeros iguales dan el promedio uniforme (softmax de iguales = uniforme).



= mismos pesos uniformes de la Versión 2

El molde que queda

Toda esta familia de operaciones es siempre la misma secuencia de cuatro pasos.

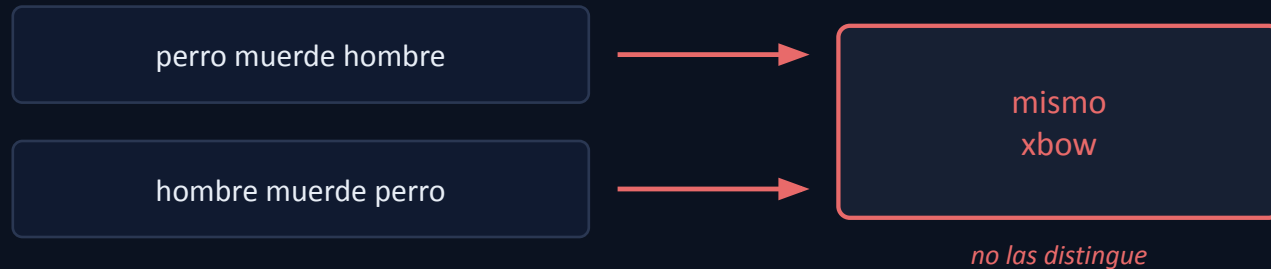


Fijá el molde: no cambia más. hasta acá, la caja 1 sale de `torch.zeros` — pesos fijos, uniformes, ciegos a los datos. Ésa es la ÚNICA pieza que vamos a cambiar.

El problema del promedio

1 · Descarta el ORDEN. un promedio no distingue {a,b,c} de {c,a,b}: las mismas palabras en distinto orden dan el mismo resultado.

2 · Descarta la RELEVANCIA. todos los tokens del pasado pesan igual, $1/(i+1)$. Un token clave pesa lo mismo que uno irrelevante.



La pregunta que abre todo: ¿y si cada token pudiera decidir de qué tokens del pasado sacar información, en vez de promediar a todos por igual? Eso es la atención.

► **VER IMPLEMENTACIÓN (notebook)** el bloque de agregación (for → matriz → softmax). No es un modelo entrenado todavía: es la pieza de “memoria” que la atención va a volver ponderada.

La memoria, ¿alcanza sola?

Probamos el bloque de agregación como modelo: embeddings → promedio causal → capa lineal → logits.

```
Andnt ik tridcowi,en  lnfebte
  airetkbobeo  etartgrta s healata
hsbar ithhe uwqorthe.tb r dtlahoate  wccramoak,fns n om otouh
nowts
mtof itih ilt mil ndilo,,
eg irweesen cin latiHntidrov ts, eBeh m pneair. eWabs

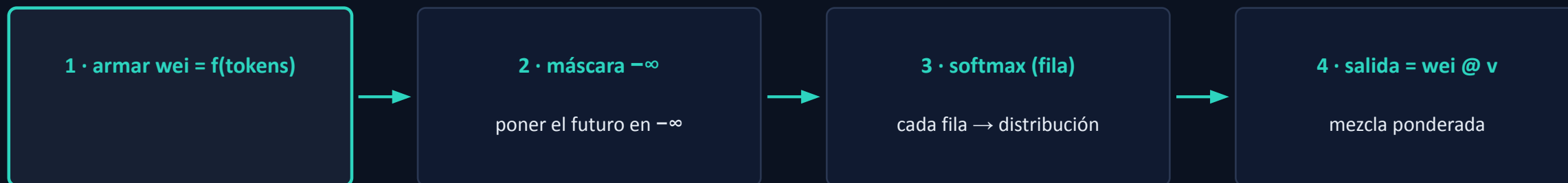
elhlind ee l.tlhu
e oncrmhy: tu
r aissphlwtye olrinfu n roopetelavle
nd:
ty wod motSakleo W-nso whdtCeibas eneti dosri, T eehhieecs  sohmower; re
het admtntruf ef sorr iok tnm:f o rey aleoontw,f f Prree d Som.trHK-
LIE!
A,NSbNI :
aadsad this ghestohuin cstk earanny  ry tsmohan yf voey
```

Observación. el promedio uniforme diluye el token actual — que es el más predictivo — así que la memoria sola no baja el loss de forma clara.

La conclusión. no queremos un promedio uniforme, queremos uno donde cada token decida cuánto pesa cada parte del pasado. Es decir: hace falta memoria PONDERADA → atención.

El objetivo: pesos que dependan del contenido

Cambiamos UNA sola pieza del molde. w_{ei} deja de salir de `torch.zeros` y pasa a calcularse a partir de los tokens. El andamiaje causal — máscara $-\infty$, softmax, $w_{ei} @ v$ — queda intacto.



↑ *ésta es la única pieza que cambia*

La meta. que cada token decida CUÁNTO atender a cada token del pasado — en vez de promediar a todos por igual.

Query, Key, Value: tres proyecciones de cada token

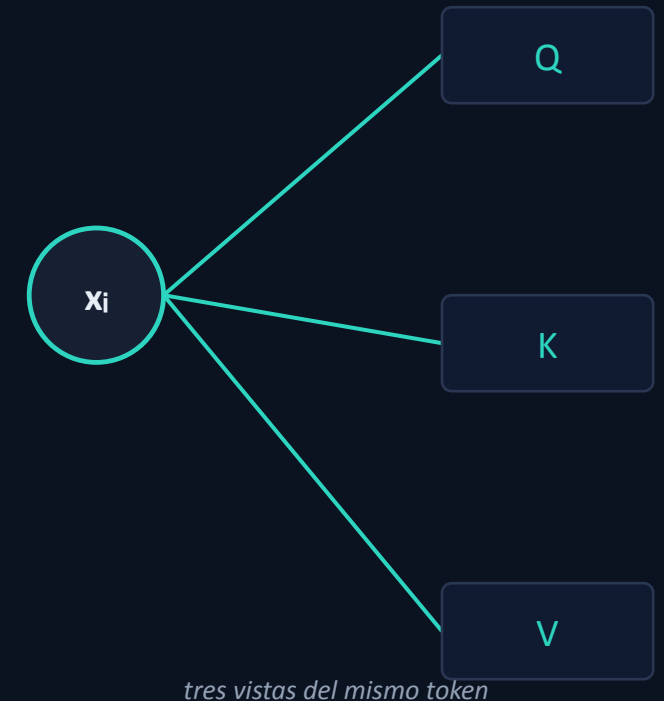
A cada token le aplicamos tres capas lineales aprendidas ($C \rightarrow \text{head_size}$):

Query $q = x \cdot W_Q$ *“qué estoy buscando”*

Key $k = x \cdot W_K$ *“qué ofrezco para ser encontrado”*

Value $v = x \cdot W_V$ *“qué apporto si me eligen”*

Lo aprendido son las matrices, no los vectores: q, k, v se recalculan en cada pasada. Son capas lineales — Wx — como cualquier otra.



La afinidad es un producto interno

$$\text{wei}[i, j] = q_i \cdot k_j$$

Alto si la query de i se alinea con la key de j. El producto interno mide alineación — un valor alto significa “el token j ofrece lo que el token i busca”.

$q @ k^T$ produce la matriz completa de afinidades $T \times T$: cada query evaluada contra cada key, de una sola vez.

token 2: su query contra cada key

$$q_2 \cdot k_0$$

4.20

$$q_2 \cdot k_1$$

4.18

$$q_2 \cdot k_2$$

1.12

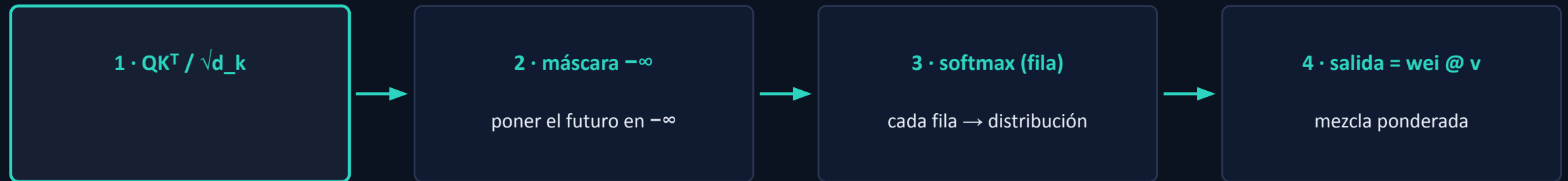
$$q_2 \cdot k_3$$

-1.42

se alinea fuerte con 0 y 1 · en contra de 3

La fórmula de una cabeza de atención

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k} + \text{máscara}) \cdot V$$



El mismo molde de la Parte 2, con la diferencia central de que ahora wei sale de las afinidades entre tokens, no de ceros. QK^T da todas las afinidades; se escala; se enmascara y normaliza con softmax; y $\cdot V$ mezcla los valores. **Entra $T \times d$, sale $T \times d$.**

El mapa de atención: pesos ya no uniformes

El pago de toda la construcción. con los pesos calculados desde el contenido, cada token distribuye su atención de forma despareja — según relevancia — en vez de repartir $1/(i+1)$ a todos.

PROMEDIO uniforme (Parte 2)

1.00	0.00	0.00	0.00
0.50	0.50	0.00	0.00
0.33	0.33	0.33	0.00
0.25	0.25	0.25	0.25

vs

ATENCIÓN aprendida

1.00	0.00	0.00	0.00
0.30	0.70	0.00	0.00
0.49	0.49	0.02	0.00
0.10	0.25	0.20	0.45

Misma forma triangular, intensidades muy distintas. fila 2: atención [0.49, 0.49, 0.02] vs uniforme [0.33, 0.33, 0.33].

Por qué la atención le gana al promedio

Relevancia

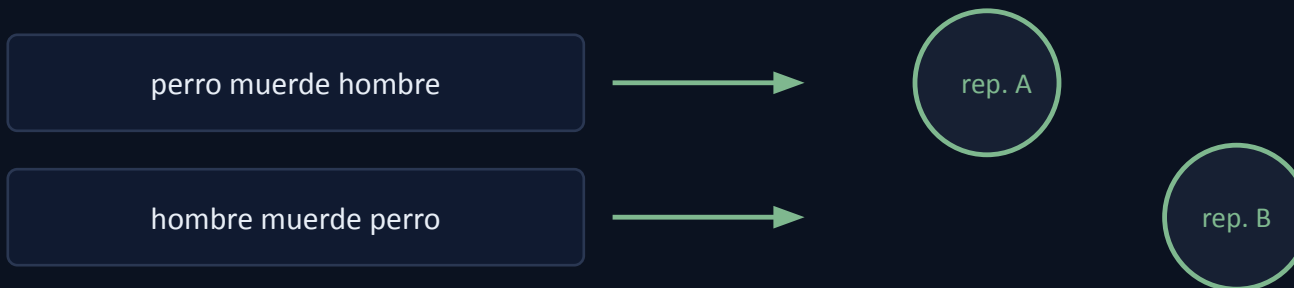
los pesos $q \cdot k$ son desparejos: un token puede pesar mucho y otro casi nada.

Orden

se recupera aparte, inyectando la posición (próxima lámina).

Value $\neq x$

la salida mezcla los VALUES, no los x crudos: routing ($q \cdot k$) y contenido (v) separados.



ahora: pesos distintos $\rightarrow$ representaciones distintas

La clave. query y key deciden a QUIÉN se atiende; el value es QUÉ se transporta. Esa separación es parte de la flexibilidad del mecanismo.

El escalado $1/\sqrt{d_k}$: argumento de varianza

con q, k de componentes indep., media 0, var 1: $\text{Var}(q^T k) = d_k$

El producto interno suma d_k términos de varianza 1, así que su varianza es d_k y su desvío $\sqrt{d_k}$: cuanto mayor la dimensión, más se dispersan los puntajes.

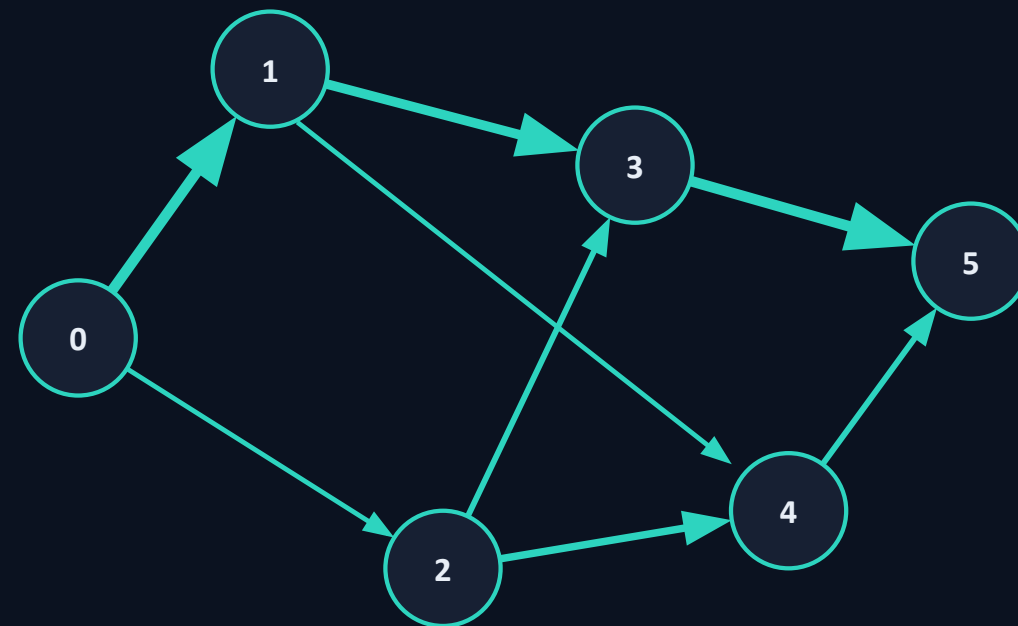
El problema. puntajes grandes llevan el softmax a saturarse (casi one-hot), y ahí el gradiente se apaga. Dividir por $\sqrt{d_k}$ renormaliza la varianza a 1 y lo mantiene en una zona con gradiente útil.



Atención como comunicación en un grafo

Los tokens son nodos. cada token agrega información de los nodos a los que atiende, con un peso por arista. La máscara causal = sólo aristas hacia atrás.

Sin noción de espacio. la atención trata a los tokens como un conjunto — no sabe distancias ni orden. Por eso la posición se agrega por separado (próxima lámina).

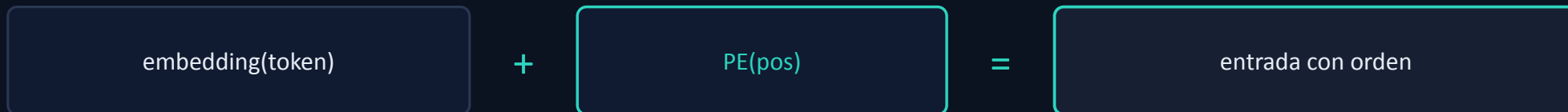


cada nodo recibe de los anteriores · grosor = peso · arcos hacia atrás

Falta la posición: positional encoding

La atención es equivariante a permutación: barajar los tokens de entrada baraja la salida, pero no cambia los valores. Es decir, es ciega al orden — y en lenguaje el orden es información.

solución: $\text{entrada} = \text{embedding}(\text{token}) + \text{PE}(\text{pos})$



Con eso, cada token lleva incorporado dónde está, y dos oraciones con las mismas palabras en distinto orden dejan de ser indistinguibles.

“Self” y una nota sobre el batch

SELF-ATTENTION

Q, K, V salen de la MISMA secuencia: la secuencia se consulta a sí misma. (Si K y V vinieran de otra secuencia, sería cross-attention — pero no es este caso.)

EL EJE DE BATCH B

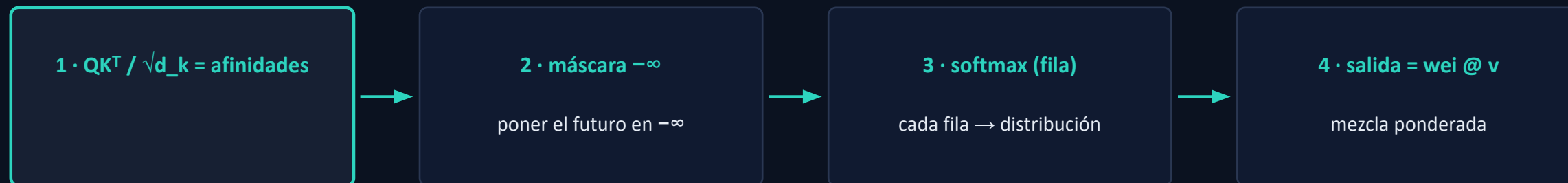
es sólo paralelismo: secuencias independientes procesadas juntas por eficiencia, que nunca se mezclan. La atención opera sobre el eje T, dentro de cada secuencia.

$x : (B, T, C)$

B = secuencias (batch size) · T = posiciones (largo de la secuencia) · C = canales (dim embedding)

Síntesis: una cabeza de atención

En una línea. cada token emite una query y una key; se puntúa cada query contra cada key; se enmascara causalmente y se normaliza con softmax; y cada token se reconstruye como el promedio ponderado de los values que encontró relevantes.



cambiamos UNA pieza del molde de la Parte 2 — el origen de wei — y el promedio uniforme se volvió atención.

► **VER IMPLEMENTACIÓN (notebook)** reemplazar la matriz de ceros por $q \cdot k^T$ en el bloque de atención. Entrenar y generar.

Bigrama + una cabeza de atención

Ahora con memoria PONDERADA (atención) + posición.

```
And thif bridcowinen is by bt madiset bobe toe.
Sagrth my dalitanss:
Wanthaf us hat vet?

Metlad ate awice my.

Hand com onou thowns, tof is he me mil;
I lin,
Wh iree sen cie lat Het drovets, and the ngan iserans!
Al lind peal.
-hul gouchiry ptupr aiss haw ye n's nes normopeeeelaves
Momy yuke tiothakeeo Windo whe Ceiiby we ati dourive cen, ire st so mower; th
To kind thrupt f so;
ARID Wam:
ET:
I inle ont ffaf Prred my om.

He-
LIERLA,
Sby ake adsad this and thinin cour ay aney Iry ts I fr yo ve y
```

Qué mirar. el loss BAJA respecto del bigrama, y el texto gana estructura: fragmentos tipo palabra, líneas, el patrón “NOMBRE:”. Sigue sin sentido, pero se parece más a un guion.

mapa de atención aprendido:

1.00	0.00	0.00	0.00
0.30	0.70	0.00	0.00
0.49	0.49	0.02	0.00
0.10	0.25	0.20	0.45

triangular, pero desparejo

Una cabeza captura una sola relación

El cuello de botella. una cabeza tiene un solo par $W_Q / W_K \rightarrow$ una sola noción de “relevante” $\rightarrow$ una única distribución de atención por token. Esa distribución debe servir a la vez para todos los tipos de relación, promediándolos.

En el lenguaje conviven varios tipos de relación al mismo tiempo — concordancia sujeto-verbo, proximidad, correferencia. Una sola distribución los mezcla en un promedio.

una sola forma de mirar el pasado:

1.00	0.00	0.00	0.00
0.30	0.70	0.00	0.00
0.49	0.49	0.02	0.00
0.10	0.25	0.20	0.45

sintaxis

proximidad

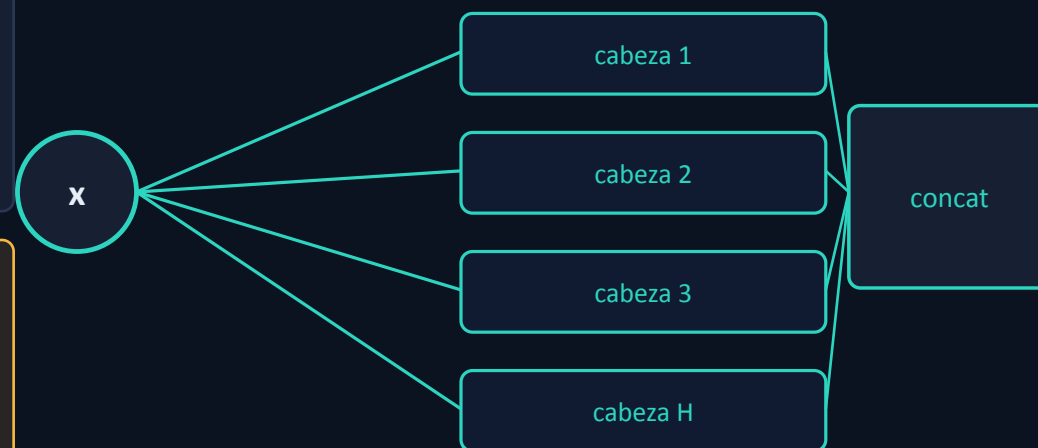
correferencia

↘ ↓ ↙ *colapsadas en una matriz*

La idea: varias cabezas en paralelo

Multi-head. varias cabezas independientes, cada una con sus propias W_Q , W_K , W_V , corriendo en paralelo sobre el mismo x . Como sus matrices son independientes, cada una calcula su propia atención.

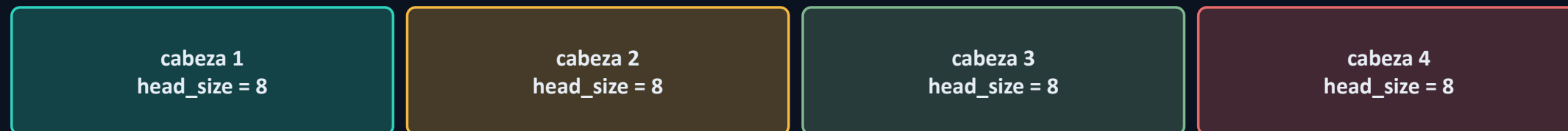
Son ramas distintas del grafo, con parámetros distintos — no un truco de cómputo. No interfieren entre sí: durante la atención no se comunican, recién se juntan en la concatenación.



El reparto de dimensión: $\text{head_size} = C / H$

No se apilan cabezas de tamaño completo: se reparte el ancho. H cabezas de tamaño C/H , concatenadas, recuperan C. Cada cabeza opera en un subespacio; la concatenación rearma el ancho.

C = 32



$4 \text{ cabezas} \times 8 = 32$ (se rearma C tras concatenar)

La forma no cambia: entra (B, T, C), sale (B, T, C). Por eso multi-head reemplaza a una sola cabeza sin tocar nada alrededor.

Ejemplo: dos cabezas, dos atenciones distintas

Mismo input, dos cabezas con proyecciones independientes. Miremos qué hace el token 3 (última fila): cada cabeza atiende a tokens distintos.

cabeza A

1.00	0.00	0.00	0.00
0.01	0.99	0.00	0.00
1.00	0.00	0.00	0.00
0.02	0.41	0.07	0.49

cabeza B

1.00	0.00	0.00	0.00
0.96	0.04	0.00	0.00
0.03	0.34	0.63	0.00
0.29	0.71	0.00	0.00

Token 3, dos distribuciones distintas: cabeza A [0.02, 0.41, 0.07, 0.49] · cabeza B [0.29, 0.71, 0, 0]. Cada cabeza hace una pregunta distinta sobre el pasado.

Cada cabeza es una pregunta propia

q, k, v son POR cabeza. un token no emite una query: emite una query por cabeza, porque cada cabeza tiene su propio W_Q . Y la key y el value también son por cabeza.

mismo token 3, dos queries distintas:

$$q_A = [0.66, 2.45, -2.77, 0.12]$$

$$q_B = [-0.30, 2.74, -6.63, 4.97]$$



Cada cabeza es un canal de comunicación independiente: cada token pregunta, ofrece y entrega algo distinto en cada canal, en paralelo.

Concatenar y proyectar

1 • Concat. las H salidas se concatenan $\rightarrow (B, T, C)$. Cada cabeza ocupa su propio bloque de canales, sin mezclarse.

2 • Proyección W_O . una capa lineal mezcla esos bloques entre sí, para que cada canal combine lo que juntaron las distintas cabezas.



Sin la proyección, las cabezas quedarían en carriles separados. W_O es lo que les permite interactuar. Entra (B,T,C), sale (B,T,C).

Costo: multi-head sale casi gratis

Parámetros de las proyecciones Q/K/V:

una cabeza tamaño C: $3 \cdot C^2$

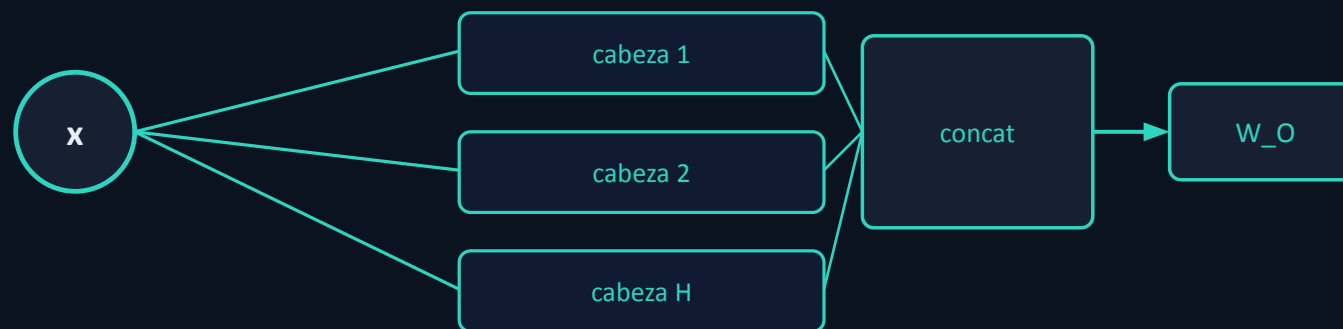
H cabezas tamaño C/H: $H \cdot 3 \cdot C \cdot (C/H) = 3 \cdot C^2$

Idéntico. dividir una cabeza grande en H chicas no cuesta más parámetros — pero da H patrones de atención independientes en vez de uno. Expresividad prácticamente gratis, sólo reorganizando los mismos pesos.



Síntesis: multi-head

En una línea. correr H cabezas independientes en subespacios de tamaño C/H , cada una con sus propias W_Q , W_K , W_V ; concatenar sus salidas y pasarlas por una proyección W_O . El mecanismo de una cabeza no cambia; se instancia H veces en paralelo.



► **VER IMPLEMENTACIÓN (notebook)** reemplazar la cabeza única por MultiHeadAttention (varias cabezas + concat + W_O). El loss baja; la generación gana estructura.

Multi-head: varias cabezas

Ahora con varias cabezas de atención en paralelo (+ proyección W_O).

SALIDA GENERADA · notebook

```
weren:
I'cp!
Fis.
Pne A way yole dome Hendadees:
Thing, alll If on netsspavend?

HARD OF:

Tert!
I I for I my's livee A me with thands;
Do
I E ERFA icCI:
A mobe loichm
Bown and not maw ss ont urt do I gor fo
BulfpPEND.

Leffalse in, I gint!,
Hall; Sawt s'd.
in to inssto old gher,
Be, as as,
Go arterrour the thad 't'thord, gord givers!
De ar illoinds himmaly eater, pono, tiour ang as's,
Tionafantheascome modust is me lotse, have,
Lich froont law anden'd corlo-siser,
Feroe as,
```

Qué mirar. el loss baja un poco más que con una sola cabeza, y el texto gana coherencia local — los fragmentos tipo palabra son más consistentes.

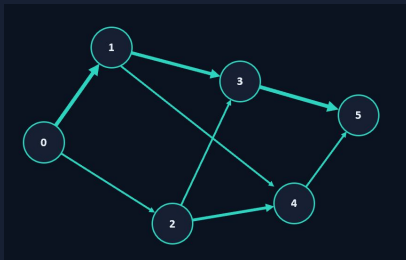
Por qué. cada cabeza captura un tipo de relación distinto en paralelo, así que el modelo integra más información útil — y todo al mismo costo de parámetros que una cabeza grande.

Falta el cómputo por token

La **atención COMUNICA tokens**, mezcla información entre posiciones. Pero después de mezclar, cada token no hace ningún cómputo propio. Falta una etapa que procese cada token por separado.

ATENCIÓN

comunicación entre tokens



¿ CÓMPUTO POR TOKEN ?

procesar cada token, solo



Esa etapa es una **pequeña red feedforward** — y es un objeto que ya conocemos.

Feedforward: cómputo por token

$$\text{FFN}(x) = W_2 \cdot \varphi(W_1 \cdot x + b_1) + b_2$$

φ = ReLU · dimensión interna típica $4 \cdot C$ · se aplica igual e independientemente a cada posición.

ATENCIÓN

comunica tokens entre sí — mezcla horizontal, a lo largo de la secuencia.

FFN

procesa cada token en profundidad — cómputo vertical, posición por posición.

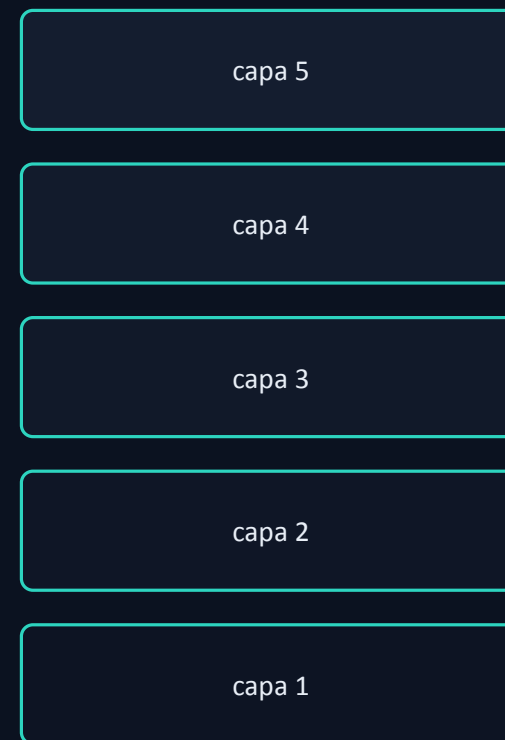
Es la unidad afín + no linealidad del principio del curso. un Transformer alterna estas dos operaciones: atención (comunicar) y FFN (procesar).

Apilar profundidad rompe el entrenamiento

Queremos apilar muchas capas. Pero apilar en profundidad, por sí solo, hace que el gradiente se desvanezca o explote al retropropagarse por muchas capas → el modelo no entrena.

Antes de apilar, hacen falta dos mecanismos que estabilicen ese flujo: las conexiones residuales y la normalización (LayerNorm).

*gradiente
que se
desvanece*



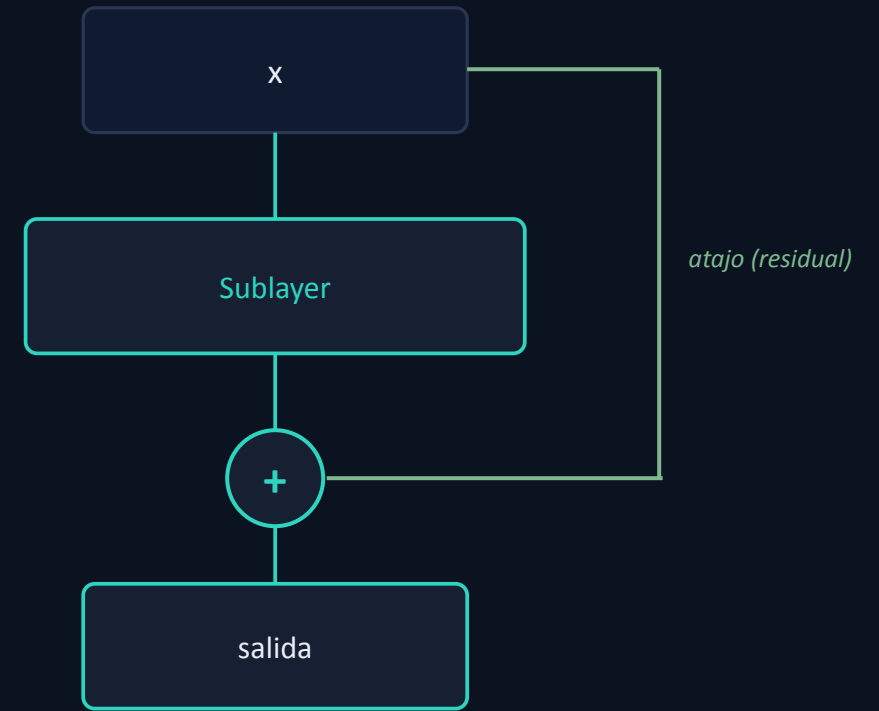
sin ayuda, la profundidad no entrena

Conexiones residuales

$$x \leftarrow x + \text{Sublayer}(x)$$

En vez de reemplazar la entrada por la salida de un sub-bloque, se la **SUMA**. Al derivar esa suma respecto de la entrada aparece un término identidad: un camino directo para que el gradiente se retropropague sin degradarse.

El ataque directo al desvanecimiento del gradiente. No cambia QUÉ calcula el bloque; cambia lo bien que se entrena en profundidad.



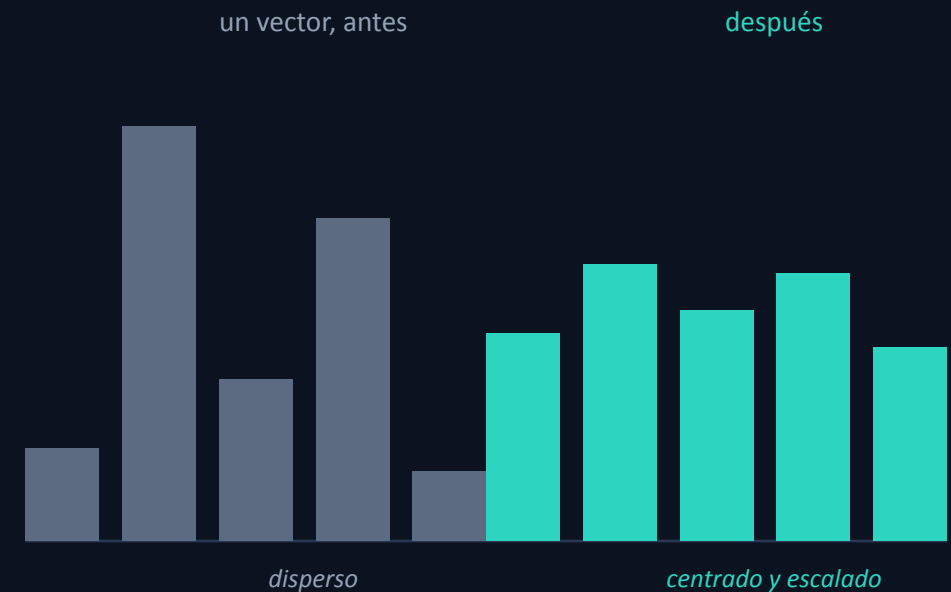
LayerNorm

Normaliza cada vector (media 0, varianza 1) sobre sus C componentes, y lo reescala con parámetros aprendidos γ , β . Mantiene las activaciones en una escala estable a través de las capas.

$$\text{LayerNorm}(x) = \gamma \cdot (x - \mu) / \sqrt{(\sigma^2 + \epsilon)} + \beta$$

μ, σ^2 media y varianza sobre las C componentes · γ, β escala y desplazamiento aprendidos · ϵ estabilidad

Por vector, no por lote. es independiente del tamaño del batch y del largo de la secuencia — ideal para texto. Junto con las residuales, hace entrenable la profundidad.



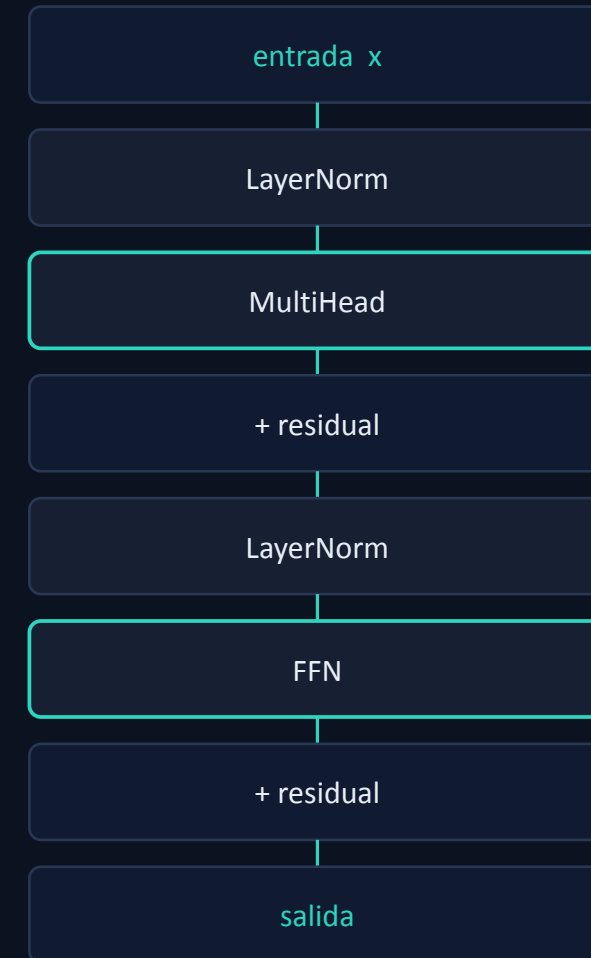
El bloque Transformer completo

Dos sub-bloques en secuencia, cada uno con residual y norm. La atención comunica; el FFN procesa; residuales y norm lo hacen entrenable.

$$x \leftarrow x + \text{MultiHead}(\text{LN}(x))$$

$$x \leftarrow x + \text{FFN}(\text{LN}(x))$$

Entra (B,T,C), sale (B,T,C). esa conservación de la forma es lo que permite el próximo paso: apilar.



Apilar N bloques: profundidad

Como cada bloque preserva la forma, se encadenan: la salida de uno alimenta al siguiente, N veces, cada bloque con sus propios parámetros. Cada capa refina la representación.



Números reales. BERT-base N=12 · GPT-3 N=96.

Capas bajas → **estructura local**; altas → relaciones abstractas. La profundidad da expresividad.

► **VER IMPLEMENTACIÓN (notebook)** envolver atención + FFN en un Block con residuales y LayerNorm, y apilar N. El loss baja marcadamente.

El modelo de lenguaje completo



Los tokens se vuelven vectores; N bloques los contextualizan; una capa lineal proyecta cada posición al tamaño del vocabulario para dar los logits del próximo token.

La diferencia con el bigrama es CÓMO se calculan los logits: ya no de una fila de tabla, sino de todo el contexto procesado por atención. La generación es autoregresiva, igual que antes.

El Transformer completo

Bloques apilados: atención + feed-forward + residuales + LayerNorm.

This withink of thy woulded morechord, eQo it the erhap,
 As master,' again me such upsuachnes great the eebity withor wirl
 Untlewn innot deat the wear not hath his coor is aniolart fre
 tengs see. O he?
 My lord, Give. To
 Comforriand, and looke of dids Engratious!
 My womay, and heeving me, I 'dill,
 Than ipguer aft humps her, here separ my mrong?
 A capwild first good a heards, gown,
 Whom sit our poldry that in the tribhers; you bed child be reforrow petteints
 Thinking and that clook ord saff,
 Which his pity away Lordy: may faid this!

ROMEO:
 I call neven the earniess trocks.

JULIET:
 That back this thy fools usford of the smaven hards.
 If Kany:
 Yet thou truth their: be habe; niene,
 Swer your hear man cumes 'More afd
 The at meagy werly daying?

Slass, I'll sent to shorm:
 And the doth betinS: what be marcy, but stand I age my king us knight swer
 That meet steath at of cuouary a goody:
 Blood my made Pires: reery of day my lipt is marrows:
 A past thou batony
 But this sobe let's my, bedry hear

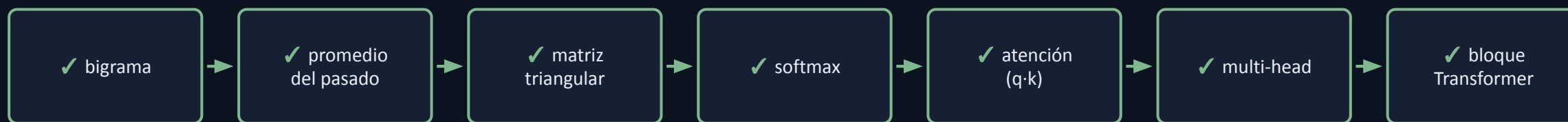
Qué mirar. el loss más bajo de toda la serie, y el texto ya con estructura clara de guion: nombres en mayúscula con dos puntos, saltos de línea, palabras plausibles.

Por qué. la profundidad (varios bloques) más el cómputo por token capturan patrones y dependencias que las versiones anteriores no podían.

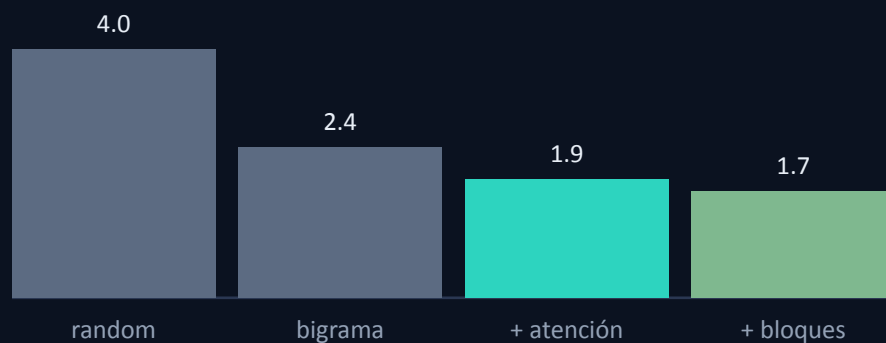
Sigue sin sentido, pero la forma es la de una obra. para acercarlo de verdad a Shakespeare, sólo falta escala (Notebook 3).

La progresión, encendida

El mapa del principio, ahora recorrido entero.



loss por etapa (referencia)



En cada paso: bajó el loss, y la estructura del texto generado se acercó más a un guion — aun sin ganar sentido. Nada del Transformer apareció de la nada.

Aclaración: construimos solo un decoder

Todo lo anterior es un Transformer decoder-only: atención causal (cada token ve solo el pasado), entrenado para predecir el próximo token. Es la arquitectura detrás de los modelos de lenguaje generativos.

LO QUE CONSTRUIMOS

**decoder
(una mitad)**

*esta sección compara
ambos*

EL PAPER (VASWANI, 2017)

**encoder + decoder
(dos mitades, para traducción)**

El problema que motiva las dos mitades: traducir

Traducir no es continuar un texto: es transformar una secuencia COMPLETA en otra. Hay dos secuencias con roles distintos: una de entrada (se lee entera) y una de salida (se genera token a token).

ENTRADA — se lee completa

the

cat

sat

SALIDA — se genera token a token

el

gato

se

sentó

Esa asimetría — una secuencia que se lee completa, otra que se genera — es lo que da lugar a las dos partes de la arquitectura.

Las dos mitades: encoder y decoder

ENCODER

Lee la entrada **COMPLETA**, con **atención bidireccional** (cada token ve todo, antes y después). Produce una representación contextual de la entrada.

DECODER

Genera la salida **token a token**, con **atención causal** (como la nuestra, para no ver el futuro). Y además **CONSULTA** la representación del encoder.

Esa consulta entre las dos secuencias es la pieza nueva: la cross-attention (próxima lámina).

La pieza nueva: cross-attention

Misma fórmula de atención; lo único que cambia es de dónde salen Q, K, V.

SELF-ATTENTION (lo nuestro)

Q, K, V salen de la MISMA secuencia.

$$Q, K, V \leftarrow x$$

CROSS-ATTENTION

Q de la secuencia que se genera (decoder); K y V de la entrada (encoder).

$$Q \leftarrow \text{decoder} \quad K, V \leftarrow \text{encoder}$$

Cada token que se genera puede mirar toda la entrada. En una traducción, cada palabra en español consulta la oración en inglés. El mecanismo es idéntico; cambia con qué se lo alimenta.

Cuándo se enmascara y cuándo no

La máscara causal no está siempre; depende de qué secuencia se mira y por qué.

encoder (self)

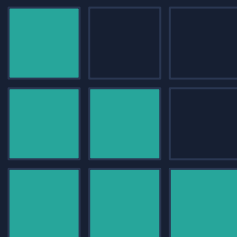
SIN máscara



la entrada se lee entera: no hay futuro que ocultar. → bidireccional

decoder (self)

CON máscara causal



al generar, no puede ver los tokens que todavía no produjo. → causal

cross (dec→enc)

SIN máscara



la entrada está completa: cada token de salida puede mirar toda la entrada.

La regla es simple: se enmascara sólo cuando ver un token sería hacer trampa respecto de lo que falta generar.

Resumen: dónde encaja lo que construimos

	atención	máscara	para qué
encoder	self, bidireccional	no	comprender una secuencia
decoder	self causal + cross	causal / no (cross)	generar condicionado a una entrada
lo que construimos	self causal	causal	generar (modelo de lenguaje)

Un mismo bloque; tres configuraciones. difieren sólo en el patrón de máscara y de qué secuencia salen Q/K/V. Lo nuestro es la versión más simple: un decoder que genera sin condicionarse a otra secuencia.

La familia completa

encoder-only

p. ej. BERT

comprensión

decoder-only

lo que construimos · p. ej. GPT

generación

← *esta clase*

encoder-decoder

el paper · p. ej. T5

**secuencia a secuencia
(traducción, resumen)**

Son tres arreglos del mismo bloque. lo que implementamos — el decoder-only — es la base de los modelos generativos, y es sobre esa base que sigue el resto del curso.

Qué queda para el curso

La arquitectura ya está completa. construida pieza por pieza a partir del modelo más simple posible. Lo que viene es entrenarla a escala y ajustarla.

- Preentrenamiento a escala (datos, cómputo).
- Fine-tuning y alineamiento sobre esta misma arquitectura.
- De ahí a los sistemas y agentes de las unidades siguientes.

La idea de la clase: cada pieza del Transformer resolvió una limitación concreta de la anterior — empezando por un bigrama que sólo contaba pares de caracteres.



Lecturas

Vaswani et al. (2017) — Attention Is All You Need. El paper original que introdujo la arquitectura Transformer. arxiv.org/abs/1706.03762

Karpathy — nanoGPT. Implementación mínima y real de un GPT. github.com/karpathy/nanoGPT

Brown et al. (2020) — Language Models are Few-Shot Learners. El paper de GPT-3. arxiv.org/abs/2005.14165